SOLAR PANEL PERFORMANCE & MAINTENANCE TRACKER

A PROJECT REPORT

Submitted by

JOBESON J

240701217

in partial fulfilment of the End Semester Examination for the course

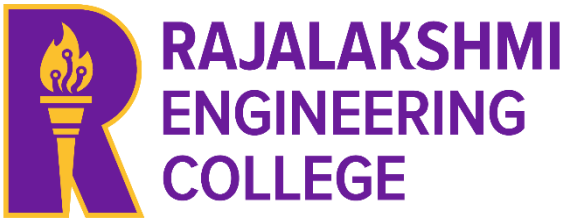
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "Solar Panel Performance & Maintenance Tracker" is the bonafide work of JOBESON J (240701217), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year[2026-2027], Semester [V], under my supervision.

SIGNATURE	**SIGNATURE**
Dr. J. MANORANJANI	BHUVANESHWARAN B
HEAD OF THE DEPARTMENT	COURSE FACULTY
Department of CSE	
Rajalakshmi Engineering College	Department of CSE
Thandalam – 602 105	Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

I JOBESON J [240701217], hereby declare that the project report titled “Solar Panel Performance & Maintenance Tracker”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: CHENNAI
Date:

SIGNATURE OF THE STUDENT
JOBESON J
240701217

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan S for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjani, Head of the Department, Department of CSE, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Bhuvaneshwaran B, Department of CSE, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

JOBESON J

ABSTRACT

The output of a solar installation depends on how closely each panel delivers the energy expected of it. When a panel under-performs, the loss continues until it is noticed and maintenance is arranged, so performance readings, panel details and maintenance history need to be kept together and checked automatically.

The Solar Panel Performance & Maintenance Tracker is a monitoring and maintenance management application built on Zoho Creator. Its data model links the Solar Panels, Energy Readigs and Maintenance Records forms through lookup fields. A Deluge form workflow calculates Performance as actual energy divided by expected energy multiplied by 100 when a reading is submitted, panels are classified as Healthy, Needs Attention, Inspection Required or Critical, and a Low Performance Alert workflow is triggered when performance falls below 70 per cent. A Maintenance Reminder workflow supports maintenance records, and a Tracker Dashboard shows total panels, panel-health tiles, performance and energy charts and maintenance status. A Custom API, Fetch Solar Weather Data, is designed to retrieve temperature, relative humidity, cloud cover and solar radiation from the free Open-Meteo API, and a Solar Performance Analyst AI agent is defined as a microservice for analysis and maintenance recommendations. A Zoho Analytics workspace was created for further analysis.

The report is limited to what the project evidence shows. The performance values on the dashboard agree with the stated formula, and the weather API call was reported successful. The Zia chat assistant was disabled for the organisation in the account screenshots, the thresholds and code shown are reference values and AI outputs were not available, roles and permissions were not defined, and the project ran in a single environment whose trial has expired.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Solar Energy Monitoring; Preventive Maintenance; Custom API; AI Agent; Open-Meteo

PROJECT AT A GLANCE

Attribute	Details
Project Title	Solar Panel Performance & Maintenance Tracker
Application Domain	Renewable energy – solar panel performance monitoring and maintenance management
Student Name & Register No.	JOBESON J [240701217
Zoho Creator Edition	Trial edition (the application header shows that the trial has expired)
Application Name (Link Name)	solar-panel-performance-maintenance-tracker (from the browser address)
Published / Live URL	https://creator.zoho.in/preview/juliusjobeson/solar-panel-performance-maintenance-tracker/ (preview address seen in the browser)
Demo Credentials (per role)	Test accounts
Number of Forms / Subforms	4 forms (Solar Panels, Energy Readigs, Maintenance Records, Self Service Form); 0 subforms seen
Number of Reports (by type)	3 named reports (All Solar Panels, Energy Readigs Report, All Maintenance Records) and 3 dashboard charts (2 column, 1 donut)
Number of Pages / Dashboards	1 dashboard page (Tracker Dashboard) and the AI Agents page
Number of Workflows	Form: 4 (Table 5.3) Schedule: 0 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	Custom API function Fetch Solar Weather Data; 4 form workflows with Deluge actions
Integrations	Open-Meteo API (Custom API); Zoho AI agent (Solar Performance Analyst); Zoho Analytics workspace; Solar Analytics Connection
Roles & Profiles Defined	None defined (single owner account)
Environments Used	Single application instance (development, stage and production not used)
Date of Production Deployment	Not deployed to a separate production environment (application created 03-OCT-2026)

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES xi
LIST OF ABBREVIATIONS xii
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
1.7 Solar Performance Concepts 3
1.8 Preventive Maintenance in Solar Plants 4
LITERATURE SURVEY AND SYSTEM STUDY 5
2.1 Study of the Existing System 5
2.2 Limitations of the Evidence Available 5
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 7
2.6 Proposed System 7
2.7 Technology Study 7
2.8 Feasibility Study 8
2.9 Risk Analysis 9
REQUIREMENT ANALYSIS 10
3.1 Stakeholders and User Roles 10
3.2 Functional Requirements 10
3.3 Non-Functional Requirements 11
3.4 Hardware and Software Requirements 12
3.5 Use Case Model 12
3.6 Module-wise Functional Specification 13
3.7 Data Dictionary 14
3.8 Additional Use Cases 15
3.9 Assumptions and Constraints 17
SYSTEM DESIGN 18

4.1 System Architecture 18
4.2 Data Model 18
4.3 Form Design 19
4.4 Process and Workflow Design 20
4.5 User Interface and Navigation Design 21
4.6 Access Control Design 21
4.7 Detailed Design of the Workflows 22
4.8 Dashboard Design 23
4.9 Integration Design – Weather Data 24
4.10 AI Agent Design 24
4.11 Data Validation and Integrity Considerations 25
IMPLEMENTATION 26
5.1 Application Setup 26
5.2 Forms and Data Capture 26
5.3 Reports 28
5.4 Pages and Dashboards 28
5.5 Deluge Scripting 30
5.6 Workflows and Automation 30
5.7 Integrations 32
5.8 User Management and Role-Based Access 33
5.9 AI and Analytics Features 33
5.10 Pro-Code Extension 34
5.11 Mobile and Portal Access 34
5.12 Application Walkthrough 35
5.13 Sample Data and Worked Calculation 35
5.14 Implementation Status Summary 36
TESTING AND DEBUGGING 38
6.1 Testing Strategy 38
6.2 Test Cases 38
6.3 Debugging Deluge Scripts 39
6.4 Defect Log 39
6.5 User Acceptance Testing 39
6.6 Planned Test Cases 39
6.7 Test Data Design 41
6.8 Recording Test Results 41
DEPLOYMENT, SECURITY AND GOVERNANCE 42
7.1 Environments and Deployment Strategy 42

7.2 Performance Optimisation 42
7.3 Security Features 43
7.4 Governance and Administration 43
7.5 Backup and Recovery Procedure 43
7.6 Risk Register 44
7.7 Operational Checklist 44
RESULTS AND DISCUSSION 46
8.1 Application Output 46
8.2 Achievement of Objectives 46
8.3 Intended Capability versus Evidence 47
8.4 Limitations of the Low-Code Approach 47
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 48
8.6 Interpretation of the Dashboard Results 48
8.7 Challenges and Lessons 48
8.8 Recommendations 49
CONCLUSION AND FUTURE ENHANCEMENTS 50
9.1 Conclusion 50
9.2 Future Enhancements 50
9.3 Learning Outcomes and Career Relevance 50
9.4 Enhancement Roadmap 51
REFERENCES 51
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 53
APPENDIX B – COURSE OUTCOME MAPPING 55
APPENDIX C – COMPLETE DELUGE SOURCE CODE 56
APPENDIX D – USER MANUAL 58
D.1 Accessing the Application 58
D.2 Registering a Solar Panel 58
D.3 Recording an Energy Reading 58
D.4 Recording Maintenance 58
D.5 Reading the Dashboard and Using the AI Agent 58
D.6 Troubleshooting 59
D.7 Glossary 59
APPENDIX E – EVALUATION RUBRIC 60
APPENDIX F – VIVA-VOCE RECORD SHEET 62

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed System....6
Table 2.2: Summary of Literature Survey....6
Table 2.3: Technologies Used and Their Role....8
Table 2.4: Project Risks and Mitigation....9
Table 3.1: Stakeholders and User Roles....10
Table 3.2: Functional Requirements....10
Table 3.3: Non-Functional Requirements....11
Table 3.4: Hardware and Software Requirements....12
Table 3.5: Use Case Description – Record an Energy Reading....13
Table 3.6: Module-wise Functional Specification....13
Table 3.7: Data Dictionary....14
Table 3.8: Use Case Description – Record a Maintenance Activity....16
Table 3.9: Use Case Description – View the Dashboard....16
Table 3.10: Use Case Description – Fetch Weather Data and Ask the AI Agent....17
Table 4.1: Entities and Relationships....19
Table 4.2: Field Specification – Fields Seen in the Forms....19
Table 4.3: Panel Classification and Alert Rules....21
Table 4.4: Design of the Calculate Performance Workflow....22
Table 4.5: Design of the Low Performance Alert Workflow....22
Table 4.6: Design of the Maintenance Reminder Workflow....23
Table 4.7: Dashboard Components....23
Table 4.8: Weather Parameters and Open-Meteo Variables....24
Table 4.9: Design of the AI Features....24
Table 4.10: Recommended Validation and Integrity Checks....25
Table 5.1: Reports and Charts Identified....28
Table 5.2: Deluge Scripts Identified....30
Table 5.3: Workflows Implemented....31
Table 5.4: Integrations Implemented....32
Table 5.5: Values Read from the Dashboard Charts....36
Table 5.6: Implementation Status Summary....36
Table 6.1: Test Cases....38
Table 6.2: Defect Log....39
Table 6.3: Planned Test Cases (Not Yet Executed)....40
Table 6.4: Test Data Design....41

Table 7.1: Deployment Checklist....42
Table 7.2: Performance-Related Design Choices....42
Table 7.3: Security Controls Identified....43
Table 7.4: Backup Components and Recovery....43
Table 7.5: Deployment and Operation Risks....44
Table 7.6: Recommended Operational Checklist....44
Table 8.1: Values Observed on the Tracker Dashboard....46
Table 8.2: Mapping of Objectives to Outcomes....46
Table 8.3: Intended Capability versus Evidence Available....47
Table 8.4: Recommendations Arising from the Review....49
Table 9.1: Enhancement Roadmap....51
Table A.1: Feature Coverage Checklist....53
Table B.1: Mapping of Course Outcomes to Report Chapters....55
Table D.1: Troubleshooting Guide....59
Table D.2: Glossary of Terms....59
Table E.1: Evaluation Rubric for the Individual Project....60
Table E.2: Performance Level Descriptors....61
Table F.1: Viva-Voce Questions and Responses....62
Table F.2: Marks Summary....62

LIST OF FIGURES

Figure 2.1: Proposed Flow for a Submitted Energy Reading 5
Figure 3.1: Use Case Diagram of the Solar Panel Tracker 12
Figure 4.1: System Architecture of the Solar Panel Tracker 18
Figure 4.2: Entity–Relationship Diagram of the Solar Panel Tracker 18
Figure 4.3: Navigation Structure of the Application 21
Figure 5.1: Zoho Creator Home Showing the Application 26
Figure 5.2: Solar Panels Form in Live Mode 27
Figure 5.3: Energy Readigs Form in Live Mode 27
Figure 5.4: Maintenance Records Form in Live Mode 27
Figure 5.5: Tracker Dashboard in Live Mode 29
Figure 5.6: Tracker Dashboard in the Page Builder 29
Figure 5.7: Form Workflows of Energy Readigs 31
Figure 5.8: Form Workflow of Maintenance Records 31
Figure 5.9: Form Workflow of Self Service Form 31
Figure 5.10: Custom API List Showing Fetch Solar Weather Data 32
Figure 5.11: Connections List Showing Solar Analytics Connection 33
Figure 5.12: Zoho Analytics Import Screen with Zoho Creator Source 33
Figure 5.13: AI Models List Showing the Solar Performance Analyst Agent 34
Figure 5.14: AI Agents Page Showing the Chat Assistant Disabled Message 34

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
DS	Deluge Script
ER	Entity–Relationship
FR	Functional Requirement
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
PV	Photovoltaic
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
WF	Workflow
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

A solar installation is judged by how much energy its panels actually deliver compared with what they are expected to deliver. When a panel delivers much less than expected, the cause may be soiling, shading, equipment faults or weather, and the loss continues until someone notices it and arranges maintenance. Keeping panel details, energy readings and maintenance history together makes such losses visible and makes maintenance easier to plan.

The Solar Panel Performance & Maintenance Tracker is a monitoring and maintenance management application built on the Zoho Creator low-code platform. It stores solar panel records, records expected and actual energy for each panel, calculates a performance percentage, classifies panel health, raises a notification when performance falls below 70 per cent, and keeps maintenance records. It also retrieves weather data from the Open-Meteo API, includes a Zoho AI agent for solar performance analysis, and shows its results on a dashboard.

1.2 Problem Statement

The project objective is to develop an intelligent monitoring and maintenance management system that tracks energy generation, identifies performance issues, automates maintenance alerts and provides AI-powered recommendations. The problems it addresses are therefore:

- Energy generated by each panel must be compared with the expected output, and the comparison must be repeatable for every reading.
- Panels that perform poorly must be identified and classified without manual calculation.
- Maintenance work (inspection, repair and the next due date) must be recorded against the panel it concerns.
- Weather conditions that influence output (temperature, humidity, cloud cover and solar radiation) should be available alongside the readings.
- Staff should be able to ask for analysis and maintenance recommendations in plain language.

The project description does not describe an earlier process, so the report treats the application as a new system and makes no claim about how panels were monitored before it.

1.3 Objectives

- Store panel information: identifier, name, location, type, capacity, installation date and status.
- Record expected and actual energy for each panel and calculate the performance percentage automatically.
- Classify each panel as Healthy, Needs Attention, Inspection Required or Critical.
- Trigger a low-performance notification when performance falls below 70 per cent.
- Maintain inspection and repair records, including the next maintenance date.
- Show total panels, panel health, performance and energy generation on an interactive dashboard.
- Retrieve temperature, relative humidity, cloud cover and solar radiation from the free Open-Meteo API through a Custom API.
- Provide AI assistance through a Zia chat agent and a Solar Performance Analyst AI agent.

1.4 Scope of the Project

In scope: three data forms (Solar Panels, Energy Readigs and Maintenance Records, the second name as spelled in the application) and a Self Service Form; form workflows for performance calculation, low-performance alerts, maintenance reminders and field show/hide rules; a Tracker Dashboard page; a Custom API that fetches weather data; a connection and an AI agent defined as microservices; and a Zoho Analytics workspace for further analysis.

Out of scope or not evidenced: roles and permission profiles, a customer portal, a mobile deployment, pro-code extensions and deployment through separate development, stage and production environments. The application was created on 03 October 2026 and its trial period had expired when the screenshots in this report were taken.

1.5 Justification for a Low-Code Approach

The application is made of forms, lookups, reports, a dashboard and a small number of rules, which are the building blocks Zoho Creator provides directly. The calculation of performance, the classification and the alerts are expressed as form workflows, and the weather interface is published as a Custom API without a separate server [3]. Zoho's AI agents are defined in the same platform [5], so the intelligent features did not require a separate machine-learning service.

The trade-offs are discussed in Chapter 8: the platform limits what can be changed (for example, the chat agent was disabled for the organisation until it is enabled under Operations → Zia), and a trial account limits how long the application stays available.

1.6 Organisation of the Report

Chapter 2 studies the problem area and the related low-code sources and states the proposed system. Chapter 3 lists the requirements and the main use case. Chapter 4 presents the architecture, data model and workflow design. Chapter 5 describes the implementation, Chapter 6 testing, Chapter 7 deployment, security and governance, and Chapter 8 the results and a discussion of the low-code approach. Chapter 9 concludes and lists future work. The appendices contain the feature checklist, course-outcome mapping, code listing notes, user manual, evaluation rubric and viva-voce record sheet.

1.7 Solar Performance Concepts

A solar panel converts sunlight into electrical energy. The energy it delivers over a period depends on the sunlight that reaches it (solar radiation), the temperature of the panel, cloud cover, the cleanliness of its surface, shading, the condition of the equipment and the age of the installation. The application deals with these through two numbers that the user supplies for every reading: the energy expected from the panel and the energy actually generated, both shown in kilowatt-hours on the dashboard charts.

The international standard IEC 61724-1 outlines equipment, methods and terminology for monitoring and analysing the performance of photovoltaic systems, and includes a performance ratio among its performance indicators [7]. The ratio used in this project is simpler: it divides actual energy by the expected energy that the user records, and multiplies the result by 100. The two are related in purpose, since both compare delivered energy with a reference, but they are not the same measure, because the

project relies on the expected value entered in each reading and not on a value derived from measured irradiation.

The expected value therefore matters as much as the measured one. A study of systems recorded by the Open Solar Performance and Reliability Clearinghouse found that, after normalising for insolation, temperature, balance-of-system efficiency and age, the sampled systems averaged 91.7 per cent of expected production, and that systems in areas with higher particulate pollution performed worse, which the authors attribute mainly to faster soiling [8]. The same study reports that the assumed shading losses affect the calculated ratio strongly [8]. These findings explain why the application keeps the expected energy as an explicit field and why weather and maintenance information are placed beside the readings.

1.8 Preventive Maintenance in Solar Plants

Maintenance in a solar plant is of two kinds: planned work done at intervals, such as inspection and cleaning, and corrective work done after a loss or a fault is noticed. The NREL study above concludes that as much as 8.3 per cent of lost production could be recovered through optimal operations and maintenance [8]. A system that records each maintenance activity, its cost, the action taken and the next due date, and that warns when performance is low, supports both kinds of work.

The Maintenance Records form of the application holds exactly these items: the problem found, the date, the technician, the cost, the action taken, the status and the next maintenance date. The Maintenance Reminder workflow and the donut chart of maintenance status (Pending, In Progress, Completed) use this record to keep planned work visible. The way the reminder is delivered was not recorded.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The project description does not describe an existing way of recording panel performance and maintenance, so the report does not assume one. What can be stated from the project itself is that the application replaces the need to calculate performance by hand: it computes Performance as Actual Energy divided by Expected Energy multiplied by 100 whenever a reading is submitted.

Figure 2.1 shows the proposed flow of a single reading through the application, from the entry of the reading to the classification and the alert decision.

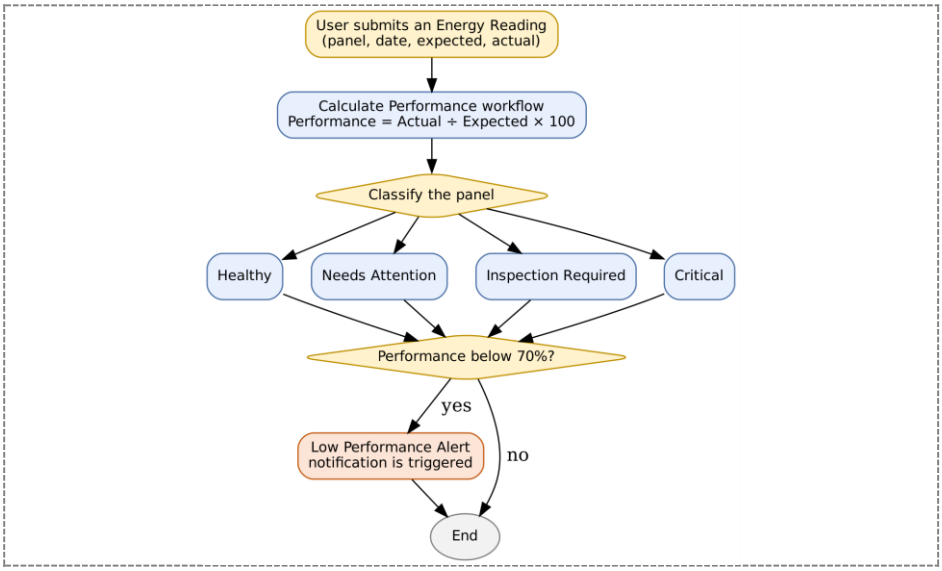


Figure 2.1: Proposed Flow for a Submitted Energy Reading

2.2 Limitations of the Evidence Available

- The classification thresholds between Healthy, Needs Attention, Inspection Required and Critical were not supplied.
- The channel of the low-performance notification (e-mail, in-app or other) was not stated.
- The Deluge source of the workflows and of the weather function was not supplied.
- No response of the AI agent and no body of the weather API response was captured.

2.3 Low-Code versus Traditional Development

Table 2.1 compares the traditional route with the route used for this application. The comparison is qualitative; no effort or performance measurements were taken.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed System

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Data model and screens	Tables, migrations and form pages are coded	Each form creates its table and data-entry page; lookups link forms
Business rules	Written in a back-end language and deployed	Written as form workflows in Deluge and run on form submission
Dashboard	Charting library and API endpoints are built	Page with chart and tile components
Outside data	HTTP client code and a server to host it	Custom API published from the platform [3]
AI features	Model hosting and integration code	AI agent defined as a microservice [5]
Control	Full control of every behaviour	Limited to what the platform offers

2.4 Literature Survey

Table 2.2 lists the sources consulted. Only sources that could be verified are included, and the summaries are limited to what the sources themselves state.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. Bock and U. Frank [1], 2021	Low-Code Platform	Discussion paper on the low-code platform concept; its keywords include citizen developer and visual development.	Frames the application as a build on a low-code platform.
2	A. Sahay et al. [2], 2020	Supporting the understanding and comparison of low-code development platforms	Technical survey of eight low-code platforms using a conceptual comparison framework; platforms are heterogeneous.	Supports checking a platform's features against requirements before choosing it.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
3	Zoho Corporation [3]	Custom APIs (Zoho Creator Help)	A Custom API exposes a function as an endpoint, created under Microservices.	Basis of the Fetch Solar Weather Data API.
4	Open-Meteo [4]	Weather Forecast API documentation	Documents hourly variables such as temperature_2m, relative_humidity_2m, cloud_cover and shortwave_radiation.	Source of the weather parameters fetched.
5	Zoho Corporation [5], [6]	Understanding AI Agent; Understanding and configuring chat agent (Zoho Creator Help)	Describe AI agents and the chat agent in Zoho Creator.	Basis of the Solar Performance Analyst agent and the chat assistant.

2.5 Emerging Trends in Low-Code Development

Two themes in the sources shaped the design. First, low-code platforms are presented as a way to build business applications visually and quickly [1], while differing in the features they offer [2], so each requirement was checked against what Zoho Creator provides. Second, the platform now includes AI agents and chat agents [5], [6], which allowed an analysis assistant to be added without leaving the platform. The report does not claim any benefit beyond what the application itself shows.

2.6 Proposed System

The proposed system is the Solar Panel Performance & Maintenance Tracker. Users register panels, record energy readings and maintenance activities, and the application calculates and classifies performance, notifies when performance is low, shows the position on a dashboard, can fetch weather data for the site and offers an AI agent for analysis and recommendations.

2.7 Technology Study

Table 2.3 lists the technologies used and the part each plays. The descriptions of the Zoho components are general; the project-specific use is taken from the application screenshots and the project description.

Table 2.3: Technologies Used and Their Role

Technology	General Description	Role in this Project
Zoho Creator	Low-code platform for building business applications from forms, reports, pages and workflows [1], [2].	Hosts the whole application on the India data centre.
Forms and lookups	A form stores records; a lookup field links a record to a record of another form.	Solar Panels, Energy Readigs and Maintenance Records, linked by Panel and Panel ID lookups.
Deluge	Zoho's scripting language for workflows and functions.	Calculates performance, classifies panels and drives the alert and reminder workflows.
Pages	Free-form screens that combine charts, tiles and other components.	Tracker Dashboard.
Custom API	Publishes a function as an endpoint that outside systems can call [3].	Fetch Solar Weather Data (GET, Public Key).
Connections	Stored authorisation to an outside service.	Solar Analytics Connection.
AI agent / chat agent	AI agents are defined under Microservices; the chat agent gives a conversational interface [5], [6].	Solar Performance Analyst agent; AI Agents chat page.
Open-Meteo API	Weather forecast service with hourly variables such as temperature_2m, relative_humidity_2m, cloud_cover and shortwave_radiation [4].	Source of temperature, humidity, cloud cover and solar radiation.
Zoho Analytics	Reporting and analysis service that can import data from Zoho Creator.	Workspace created; import screen reached.

2.8 Feasibility Study

Technical feasibility: every component the objectives call for (forms, lookups, calculation on submission, notification, dashboard, outside API call and AI agent) is available in the platform, and the screenshots show each of them in use except the delivery of notifications and AI output. Operational feasibility: the data entry is limited to three short forms whose fields are labelled in plain words, so users need little training. Cost and schedule: the project material does not state the plan cost or the effort, so no figure is given. The application ran on a trial plan, no paid licence is recorded, and the time spent was not recorded.

2.9 Risk Analysis

Table 2.4: Project Risks and Mitigation

Risk	Effect	Mitigation
Trial period ends	The application stops being available (it had expired in the screenshots).	Export the application and data; upgrade the plan or re-create the application.
Chat agent disabled for the organisation	Conversational analysis cannot be used.	Enable the feature under Operations → Zia, then test and record sample answers.
Classification thresholds not fixed in the documentation	Panels may be classified differently from what the owner intends.	Record the thresholds in Table 4.3 and in the workflow.
Dashboard tile counts something other than panels	Critical tile (22) exceeds the 7 panels, which can mislead readers.	Check the tile's source and criteria and relabel or recount.
No roles or permissions	Every user with access sees and can change all records.	Define roles and permission profiles.
Public Key on the Custom API	Anyone holding the key can call the API.	Keep the key private; move to OAuth if the data becomes sensitive.
Weather data not stored with readings	Weather cannot be compared with performance later.	Store the fetched values in a field or form for each reading.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

The project description names no user roles. Table 3.1 therefore lists the parties that appear in the application itself, and the roles the application should define are left for the author to confirm.

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Application user (plant operator)	The person who enters panels, readings and maintenance records and reads the dashboard; no named roles were defined	Register panels, enter energy readings, read the dashboard and use the AI agent
Technician	Recorded in the Technician field of a maintenance record	Carries out the maintenance recorded in Maintenance Records
AI agent	Solar Performance Analyst, defined as a microservice	Analyses generation and maintenance information and recommends actions

3.2 Functional Requirements

Table 3.2 lists the functional requirements derived from the project objective and modules. In the Status column, Built means the item appears in a screenshot, Reported means it is stated in the project description but not shown in a screenshot, and Not built or Disabled records what the evidence shows.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Status
FR-01	Store panel information: ID, name, location, type, capacity, installation date and status.	User	Built
FR-02	Record expected and actual energy for a panel on a reading date.	User	Built
FR-03	Calculate Performance = Actual Energy ÷ Expected Energy × 100 when a reading is submitted.	System	Built
FR-04	Classify panels as Healthy, Needs Attention, Inspection Required or Critical.	System	Reported
FR-05	Trigger a low-performance notification when performance falls below 70%.	System	Built

FR ID	Requirement	Role(s)	Status
FR-06	Record maintenance: problem, date, technician, cost, action taken, status and next maintenance date.	User	Built
FR-07	Send a maintenance reminder when a maintenance record is submitted.	System	Built
FR-08	Show total panels, panel-health tiles, performance and energy charts and maintenance status on a dashboard.	User	Built
FR-09	Show average performance percentage on the dashboard.	User	Reported
FR-10	Retrieve temperature, humidity, cloud cover and solar radiation through a Custom API.	User	Built
FR-11	Provide a Solar Performance Analyst AI agent that analyses generation, critical panels and maintenance history.	User	Built
FR-12	Provide a chat assistant for questions on panel performance and maintenance.	User	Disabled
FR-13	Hold AI maintenance recommendations and keywords in the reading and maintenance forms.	System	Built
FR-14	Limit access by role and profile.	Admin	Not shown
FR-15	Analyse energy trends in Zoho Analytics.	User	Partly

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement and Evidence
NFR-01	Usability	Forms use dropdowns, a date picker and numeric fields, as seen in the form screenshots.
NFR-02	Responsiveness	The application preview offers desktop, tablet and mobile views; behaviour on those views was not checked.
NFR-03	Integration security	The weather Custom API is protected with a Public Key, as shown in the Custom API list.
NFR-04	Maintainability	Workflows have descriptive names and one or a few actions each.
NFR-05	Availability	The application is available while the trial lasts; the header shows that the trial has expired.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator, India data centre (creator.zoho.in)
Scripting	Deluge
AI	Zoho AI agent (Solar Performance Analyst) and Zia chat assistant
External API	Open-Meteo weather API [4]
Analytics	Zoho Analytics workspace (Free Edition)
Client device	Any computer with a web browser and Internet access
Test data	Seven panels; panel identifiers SP001 to SP006 appear in the dashboard charts

3.5 Use Case Model

Figure 3.1 shows the actors and the main use cases. Table 3.5 describes the central use case.

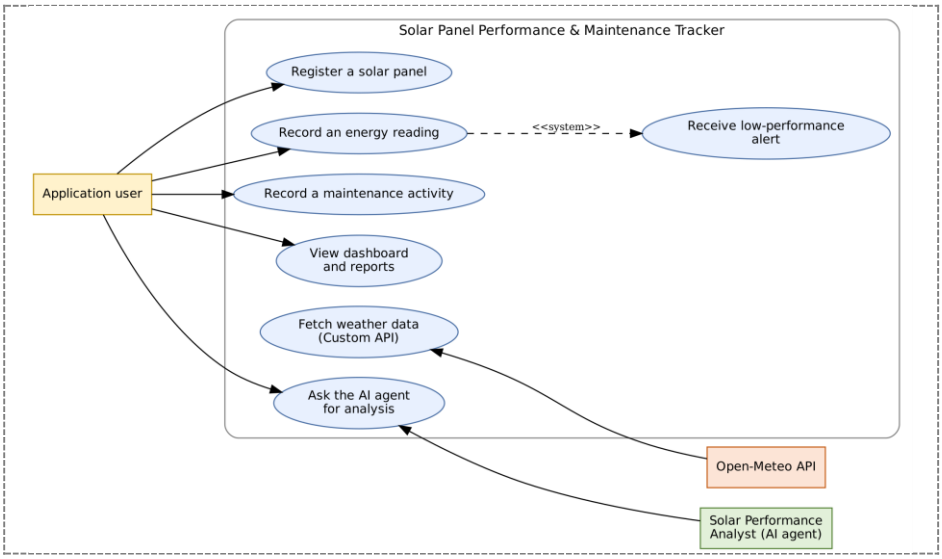


Figure 3.1: Use Case Diagram of the Solar Panel Tracker

Table 3.5: Use Case Description – Record an Energy Reading

Use Case ID	UC-01
Use Case Name	Record an energy reading and check performance
Actors	Application user (primary)
Pre-conditions	The panel exists in Solar Panels.
Main Flow	1. The user opens the Energy Readigs form. 2. The user selects the panel and enters the reading date, expected energy and actual energy. 3. The user submits the form. 4. The Calculate Performance workflow computes Performance = Actual ÷ Expected × 100. 5. The panel is classified (thresholds in Table 4.3). 6. The Low Performance Alert workflow notifies when performance is below 70%.
Post-conditions	The reading is stored with its performance value and appears in the reports and dashboard charts.
Exceptions	The behaviour when expected energy is zero or blank was not recorded; Section 4.11 recommends a check.

3.6 Module-wise Functional Specification

Table 3.6 describes each module in terms of its inputs, processing and outputs. The Evidence column shows whether the behaviour was seen in a screenshot or only stated in the project description.

Table 3.6: Module-wise Functional Specification

Module	Inputs	Processing	Outputs	Evidence
Solar Panels management	Panel ID, name, location, type, capacity, installation date, status	Stored as one record per panel	Panel list; lookup source for readings and maintenance	Form seen
Energy Readings	Panel, reading date, expected energy, actual energy	Performance = Actual ÷ Expected × 100; classification; alert below 70%	Performance value, status, notification	Form and workflows seen
Maintenance Management	Maintenance ID, panel, problem, date, technician, cost, action, status, next date	Stored per panel; reminder workflow on submission	Maintenance history; reminder	Form and workflow seen

Module	Inputs	Processing	Outputs	Evidence
Dashboard	Records of the three forms	Charts and counts	Performance and energy charts, maintenance donut, tiles	Seen
Weather integration	Site latitude and longitude as parameters of the call (required by Open-Meteo [4])	GET request to Open-Meteo through the Custom API	Temperature, humidity, cloud cover, solar radiation	API entry seen; test reported
AI assistance	Panel, reading and maintenance information	AI agent analysis	Recommendations and keywords	Agent entry seen; output not seen

3.7 Data Dictionary

Table 3.7 explains the fields of the three data forms. The meaning is stated in plain words from the field labels and the usage described in the project; units are given only where they appear in the screenshots (energy in kWh, performance as a percentage).

Table 3.7: Data Dictionary

Form	Field	Meaning	Unit / Format
Solar Panels	Panel ID	Unique identifier of the panel, such as SP001	Text
	Panel Name	Name by which the panel is known	Text
	Location	Where the panel is installed	Text
	Panel Type	Kind of panel, chosen from a list	Dropdown
	Capacity	Rated capacity of the panel	Not shown (kW is typical for panel ratings)
	Installation Date	Date the panel was installed	dd-MMM-yyyy
	Status	Current state of the panel, chosen from a list	Dropdown
Energy Readigs	Panel	The panel to which the reading belongs	Lookup
	Reading Date	Date of the reading	dd-MMM-yyyy
	Expected Energy	Energy the panel should have generated	kWh (chart axis)

Form	Field	Meaning	Unit / Format
	Actual Energy	Energy the panel actually generated	kWh (chart axis)
	Performance	Actual ÷ Expected × 100	Per cent
	Status	Health class of the reading	Dropdown
	AI Maintenance Recommendations	Text produced by AI analysis	Multi-line text
Maintenance Records	Maintenance ID	Identifier of the maintenance record	Text
	Panel ID	Panel that was maintained	Lookup
	Problem	Fault or reason for the work	Multi-line text
	Maintenance Date	Date of the work	dd-MMM-yyyy
	Technician	Person who did the work	Text
	Cost	Cost of the work	Currency not shown
	Action Taken	What was done	Multi-line text
	Status	Progress of the work: Pending, In Progress or Completed (donut chart)	Dropdown
	Next Maintenance Date	Date the next work is due	dd-MMM-yyyy
	AI Maintenance Keywords	Keywords produced by AI analysis	Multi-line text

3.8 Additional Use Cases

Tables 3.8 to 3.10 describe three further use cases. They describe the intended flows; the steps that depend on information not supplied are marked.

Table 3.8: Use Case Description – Record a Maintenance Activity

Use Case ID	UC-02
Actors	Application user; technician (named in the record)
Pre-conditions	The panel exists in Solar Panels.
Main Flow	1. The user opens Maintenance Records. 2. The user selects the Panel ID and describes the Problem. 3. The user enters the Maintenance Date, Technician, Cost and Action Taken. 4. The user chooses the Status and the Next Maintenance Date. 5. The user submits the form. 6. The Maintenance Reminder workflow runs (delivery channel not recorded).
Post-conditions	The record appears in All Maintenance Records and in the maintenance status donut.

Table 3.9: Use Case Description – View the Dashboard

Use Case ID	UC-03
Actors	Application user
Pre-conditions	Readings and maintenance records exist.
Main Flow	1. The user opens Home. 2. The Tracker Dashboard shows performance by panel, actual against expected energy, maintenance status and the panel tiles. 3. The user reads the tiles to see how many panels are healthy, need attention, need inspection or are critical.
Post-conditions	The user knows which panels need attention.
Exceptions	The Critical tile counts more than the number of panels; see Section 5.4.

Table 3.10: Use Case Description – Fetch Weather Data and Ask the AI Agent

Use Case ID	UC-04
Actors	Application user; Open-Meteo API; Solar Performance Analyst agent
Pre-conditions	The Custom API is Ready to invoke; the AI agent is added; the chat feature is enabled for the organisation.
Main Flow	1. The user (or a caller holding the Public Key) calls Fetch Solar Weather Data with the GET method. 2. The API requests temperature, humidity, cloud cover and solar radiation from Open-Meteo. 3. The user asks the AI agent about under-performing panels or maintenance. 4. The agent answers with an analysis and recommendations.
Exceptions	The chat feature was disabled for the organisation in the screenshots, so step 3 could not be carried out.

3.9 Assumptions and Constraints

- Expected energy is entered by the user for each reading; the application does not calculate it from weather data.
- One reading belongs to one panel, and one maintenance record belongs to one panel.
- The application runs on a Zoho Creator account on the India data centre and is subject to that plan's limits.
- The weather API is a free, public service; its availability is outside the control of the project.
- The AI features depend on the organisation-level setting for the chat agent.
- No roles were defined, so all users are treated alike.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The application runs entirely on Zoho Creator (Figure 4.1). Users reach it through a browser. Forms, reports and the dashboard form the presentation layer; form workflows written in Deluge form the logic layer; Creator's cloud database holds the data. Microservices connect the application to outside services: a Custom API for weather data, a connection, and an AI model that defines the AI agent. A Zoho Analytics workspace exists for analysis of the data.

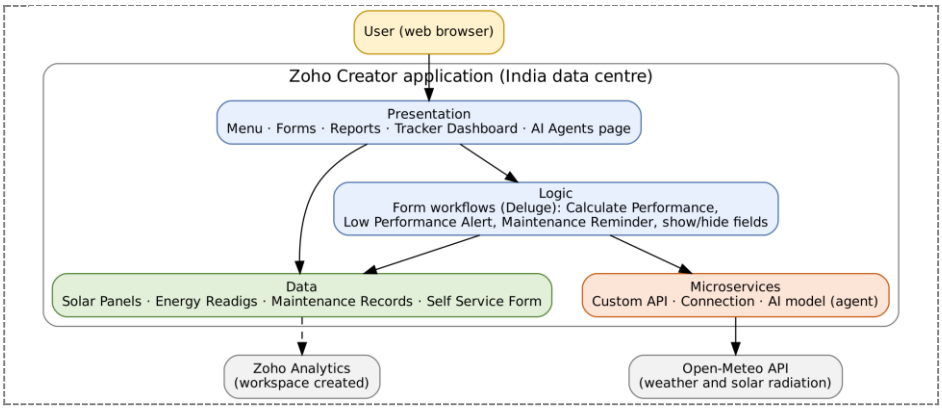


Figure 4.1: System Architecture of the Solar Panel Tracker

4.2 Data Model

The data model has three data forms linked by lookups (Figure 4.2): each reading and each maintenance record refers to one panel. A fourth form, Self Service Form, appears in the workflow list; its fields were not recorded. Table 4.1 lists the forms.

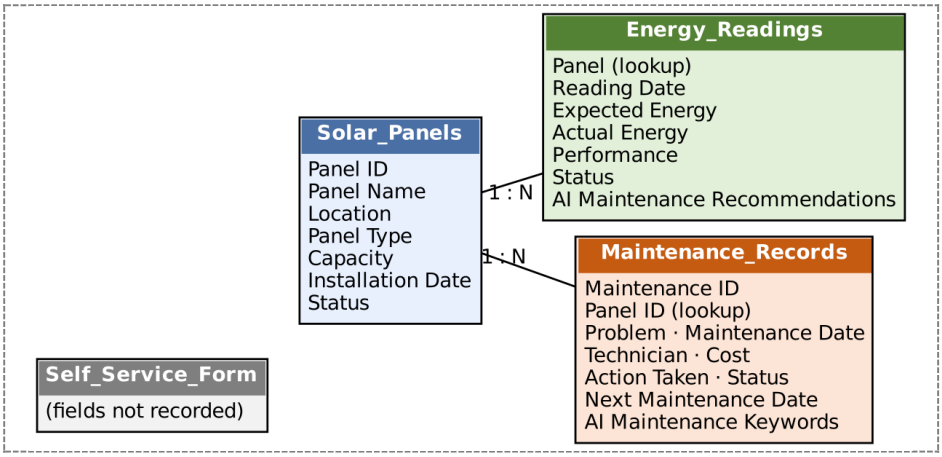


Figure 4.2: Entity–Relationship Diagram of the Solar Panel Tracker

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Solar Panels	Panel ID, Panel Name, Location, Panel Type, Capacity, Installation Date, Status	Energy Readigs; Maintenance Records	One-to-Many	Lookups in child forms
Energy Readigs	Panel, Reading Date, Expected Energy, Actual Energy, Performance, Status	Solar Panels	Many-to-One	Lookup field Panel
Maintenance Records	Maintenance ID, Panel ID, Problem, Maintenance Date, Technician, Cost, Status, Next Maintenance Date	Solar Panels	Many-to-One	Lookup field Panel ID
Self Service Form	Not recorded	None shown	—	Form with field rules

4.3 Form Design

Table 4.2 lists the fields seen in the live forms. Field types are read from the form screenshots (single-line boxes, dropdowns, date fields shown as dd-MMM-yyyy, numeric fields shown as ########.## and multi-line boxes). Field link names and mandatory settings are not visible in the screenshots and are therefore not listed.

Table 4.2: Field Specification – Fields Seen in the Forms

Form	Field Label	Field Type	Observed Input	Remarks
Solar Panels	Panel ID	Single line	Text	—
	Panel Name, Location	Single line	Text	—
	Panel Type, Status	Dropdown	-Select-	Options not shown
	Capacity	Decimal	########.##	—
	Installation Date	Date	dd-MMM-yyyy	—

Form	Field Label	Field Type	Observed Input	Remarks
Energy Readigs	Panel	Lookup	-Select-	Refers to Solar Panels
	Reading Date	Date	dd-MMM-yyyy	—
	Expected Energy, Actual Energy, Performance	Decimal	#######.##	Performance is calculated by a workflow
	Status	Dropdown	-Select-	Options not shown
	AI Maintenance Recommendations	Multi line	Text area	For AI output
Maintenance Records	Maintenance ID, Technician	Single line	Text	—
	Panel ID	Lookup	-Select-	Refers to Solar Panels
	Problem, Action Taken	Multi line	Text area	—
	Maintenance Date, Next Maintenance Date	Date	dd-MMM-yyyy	—
	Cost	Decimal	#######.##	—
	Status	Dropdown	-Select-	Options not shown
	AI Maintenance Keywords	Multi line	Greyed text area	Appears non-editable; for AI output

4.4 Process and Workflow Design

Figure 2.1 showed the flow of a reading. Table 4.3 states the rules. The class thresholds are design values adopted in this report: they agree with the 70% alert limit and place SP001 (about 82%) in the Healthy class. The thresholds actually configured in the workflow were not supplied, so if they differ, Table 4.3 and Appendix C should be changed to match.

Table 4.3: Panel Classification and Alert Rules

Rule	Condition	Result
Performance	Actual Energy ÷ Expected Energy × 100	Stored in Performance
Healthy	Performance 80% or above	Panel classified Healthy
Needs Attention	Performance 70% to below 80%	Panel classified Needs Attention
Inspection Required	Performance 50% to below 70%	Panel classified Inspection Required
Critical	Performance below 50%	Panel classified Critical
Low-performance alert	Performance below 70%	Low Performance Alert workflow notifies (channel not recorded)

4.5 User Interface and Navigation Design

The application menu (Figure 4.3) has Home, All Reports, Add New, Solar Panels, Energy Readigs, Maintenance Records and AI Agents. Home is the Tracker Dashboard. Each data form has a sub-menu that holds the data-entry form and a report.

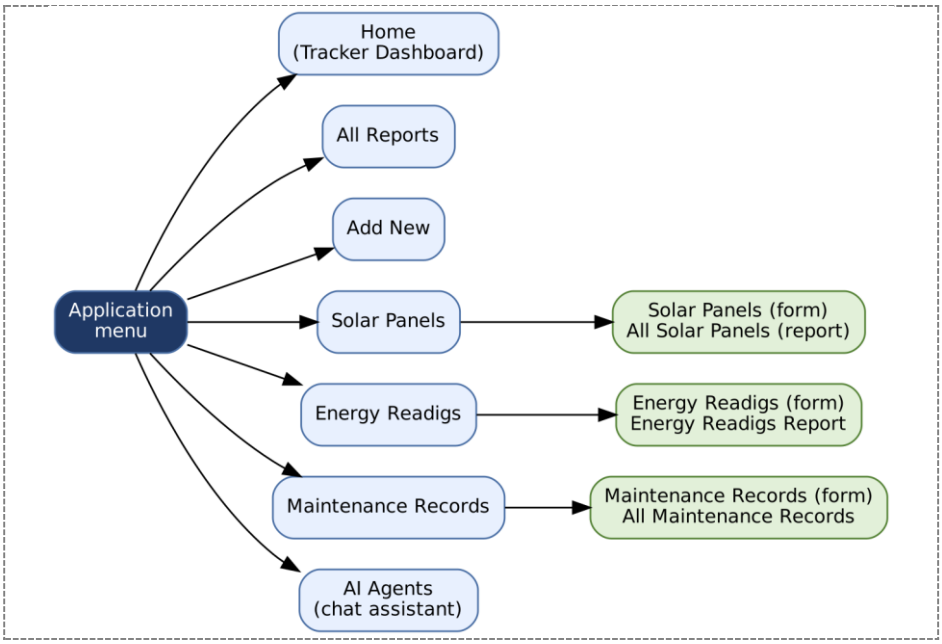


Figure 4.3: Navigation Structure of the Application

4.6 Access Control Design

No roles, permission profiles or record-level access rules were described for this application, and none is visible in the screenshots. The application therefore relies on

the default access of the Zoho Creator account that owns it. Defining roles for plant operators, technicians and viewers is listed as future work in Chapter 9.

4.7 Detailed Design of the Workflows

Tables 4.4 to 4.6 give the design specification of the three workflows that act on readings and maintenance records. The trigger and the number of actions are taken from the workflow list; the logic is the design intent from the project description, and the missing parts are marked.

Table 4.4: Design of the Calculate Performance Workflow

Attribute	Specification
Form and event	Energy Readigs – created or edited, on successful form submission
Inputs	Expected Energy, Actual Energy
Logic	Performance = Actual Energy ÷ Expected Energy × 100; the panel is then classified as Healthy, Needs Attention, Inspection Required or Critical thresholds of Table 4.3
Output	Performance value and status stored on the reading
Actions (seen)	1 action
Design consideration	The calculation needs a check that Expected Energy is greater than zero; whether the workflow contains it is not shown.

Table 4.5: Design of the Low Performance Alert Workflow

Attribute	Specification
Form and event	Energy Readigs – created or edited, on successful form submission
Condition	Performance below 70%
Output	A low-performance notification delivery channel and recipients not recorded
Actions (seen)	1 action
Design consideration	Because the workflow runs on edit as well as on creation, an edited reading could send the alert again; this was not examined.

Table 4.6: Design of the Maintenance Reminder Workflow

Attribute	Specification
Form and event	Maintenance Records – created or edited, on successful form submission
Logic	Not described in the project material; the Next Maintenance Date is the natural basis
Output	A reminder content and recipients not recorded
Actions (seen)	1 action
Design consideration	The form holds a Next Maintenance Date, which is the natural basis for a reminder.

4.8 Dashboard Design

Table 4.7: Dashboard Components

Component	Type	Source Form	Shows
Performance chart	Column	Energy Readigs	Performance % for each Panel ID
Energy chart	Column	Energy Readigs	Actual and expected energy (kWh) for each Panel ID
Maintenance chart	Donut	Maintenance Records	Records by status: Pending, In Progress, Completed
Total Solar Panels	Count tile	Solar Panels	Number of panels (7 in the screenshot)
Healthy Panels	Count tile	Not shown (D-02)	Panels classified Healthy (1)
Critical	Count tile	Not shown (D-02)	Records or panels classified Critical (22)
Inspection Required	Count tile	Not shown (D-02)	Inspection Required (3)
Needs Attention	Count tile	Not shown (D-02)	Value not visible

The layout follows the page builder view: three charts stacked on the left and five tiles in a column on the right, displayed as cards. The page supports desktop, tablet and mobile views.

4.9 Integration Design – Weather Data

The Custom API Fetch Solar Weather Data is a GET endpoint protected by a Public Key. It is designed to retrieve four environmental parameters. Table 4.8 maps each to the variable name documented by Open-Meteo [4].

Table 4.8: Weather Parameters and Open-Meteo Variables

Parameter (project)	Open-Meteo variable	Why it matters for a panel
Temperature	temperature_2m	Panel output usually falls as the panel gets hotter.
Relative humidity	relative_humidity_2m	Indicates moisture in the air and likely cloud or dew.
Cloud cover	cloud_cover	Clouds reduce the sunlight that reaches the panel.
Solar radiation	shortwave_radiation	The sunlight available to be converted into energy.

The request pattern published by Open-Meteo is shown in Appendix C. The project's own function, its coordinates and the response it returned were not supplied.

4.10 AI Agent Design

Table 4.9: Design of the AI Features

Feature	Intended Input	Intended Output
Solar Performance Analyst (agent)	Energy generation, critical panels, maintenance history, environmental conditions	Analysis and maintenance recommendations
Zia chat assistant	A request typed in plain language	Answers on panel performance, under-performing panels, maintenance records and solar monitoring questions
AI Maintenance Recommendations field	Reading and panel information	Text recommendation stored with the reading
AI Maintenance Keywords field	Maintenance record text	Keywords stored with the record

Because AI text is advisory, it should be shown as such and should never replace the calculated performance and the stored status. The way the two AI fields are populated was not recorded.

4.11 Data Validation and Integrity Considerations

The screenshots do not show validation rules on the forms. Table 4.10 lists checks that the design should include; they are recommendations, not features of the present application.

Table 4.10: Recommended Validation and Integrity Checks

Area	Check	Reason
Energy Readigs	Expected Energy greater than zero	The formula divides by it.
Energy Readigs	Actual Energy not negative	Negative generation is not meaningful.
Energy Readigs	One reading per panel per reading date	Avoids duplicate values in the charts.
Energy Readigs	Reading Date not in the future	Readings record past generation.
Solar Panels	Panel ID unique	Panel ID identifies the panel in lookups and charts.
Maintenance Records	Next Maintenance Date not before Maintenance Date	Keeps the schedule meaningful.
Maintenance Records	Cost not negative	Cost is an amount spent.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application Solar Panel Performance & Maintenance Tracker was created in Zoho Creator on 03 October 2026 (Figure 5.1); its workflows carry the same creation date. The application link name seen in the browser address is solar-panel-performance-maintenance-tracker, and the application runs on the India data centre (creator.zoho.in). The application header shows that the trial period has expired.

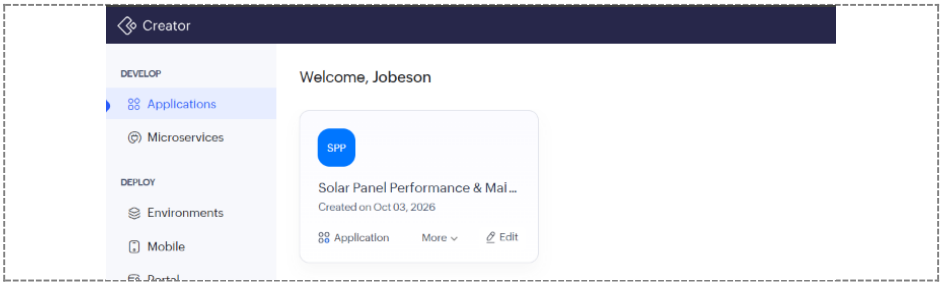


Figure 5.1: Zoho Creator Home Showing the Application

5.2 Forms and Data Capture

The Solar Panels form (Figure 5.2) captures Panel ID, Panel Name, Location, Panel Type, Capacity, Installation Date and Status. The Energy Readigs form (Figure 5.3) captures the panel, reading date, expected energy, actual energy, performance, status and a text area for AI maintenance recommendations. The Maintenance Records form (Figure 5.4) captures the maintenance ID, panel, problem, date, technician, cost, action taken, status, next maintenance date and a greyed-out AI Maintenance Keywords area. The form and menu name appears in the application as "Energy Readigs"; the report reproduces it as it is shown.

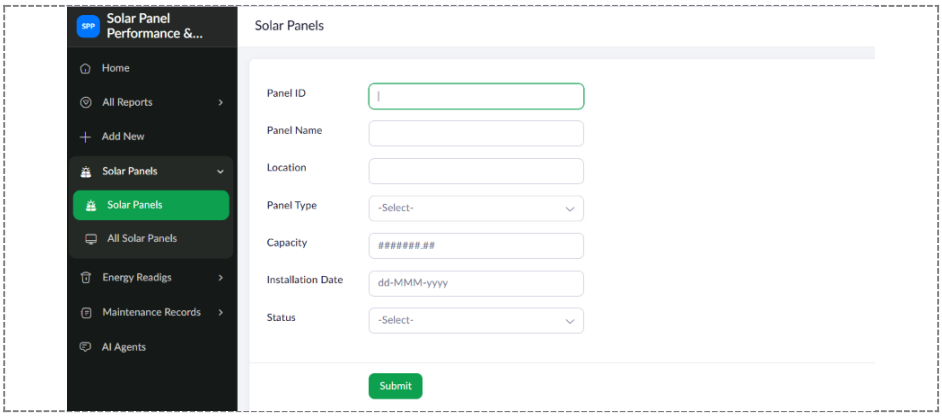


Figure 5.2: Solar Panels Form in Live Mode

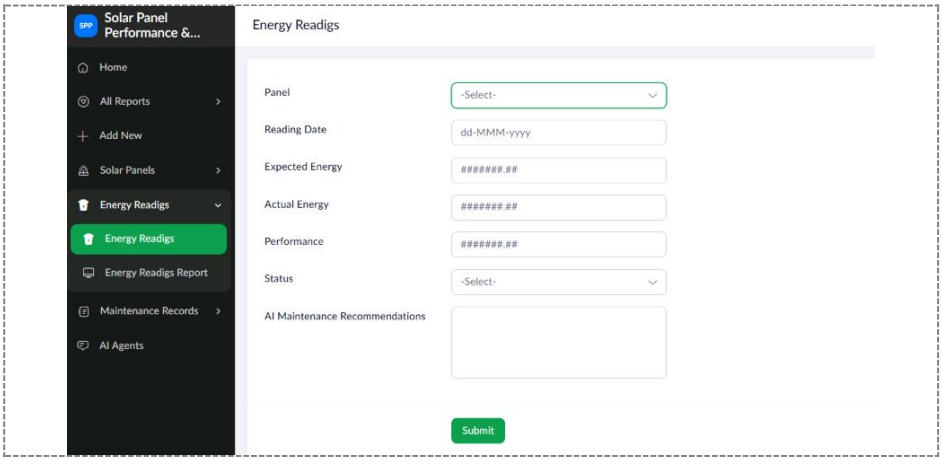


Figure 5.3: Energy Readigs Form in Live Mode

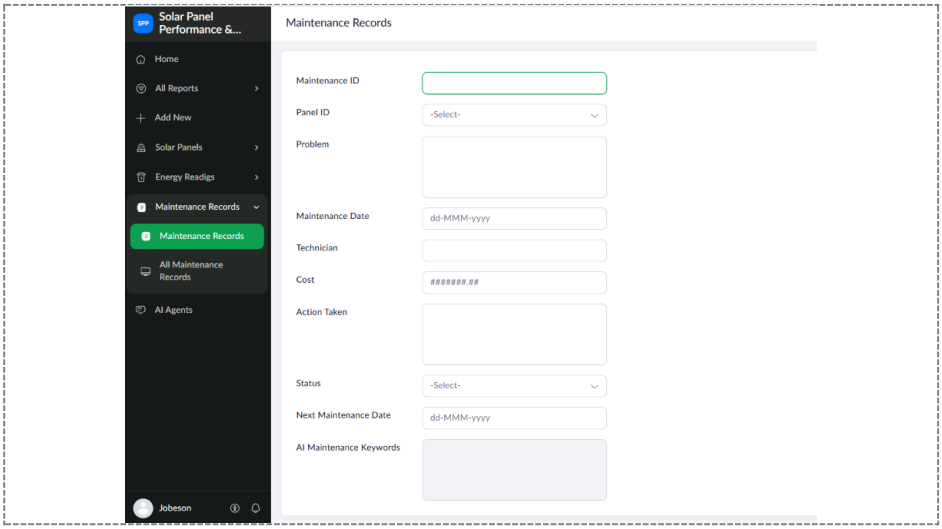


Figure 5.4: Maintenance Records Form in Live Mode

A fourth form, Self Service Form, appears in the workflow list with a field-rule workflow. Its fields and purpose were not recorded.

5.3 Reports

Table 5.1 lists the reports and charts that can be identified from the screenshots. The report type, filters and custom actions were not visible and are not claimed; the exact number of reports should be read from the application.

Table 5.1: Reports and Charts Identified

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Solar Panels	Not shown	Solar Panels	Not shown	Not shown	Not defined
Energy Readigs Report	Not shown	Energy Readigs	Not shown	Not shown	Not defined
All Maintenance Records	Not shown	Maintenance Records	Not shown	Not shown	Not defined
Performance chart	Column	Energy Readigs	Performance % by Panel ID	Not shown	Not defined
Energy chart	Column	Energy Readigs	Actual and Expected Energy by Panel ID	Not shown	Not defined
Maintenance status chart	Donut	Maintenance Records	Pending, In Progress, Completed	Not shown	Not defined

5.4 Pages and Dashboards

The Home page is a page named Tracker Dashboard (Figure 5.5). Its left column holds three charts: Performance % by Panel ID, Actual Energy against Expected Energy by Panel ID, and a donut of maintenance records by status (Pending, In Progress and Completed). Its right column holds count tiles labelled Total Solar Panels, Healthy Panels, Critical, Inspection Required and Needs Attention (Figure 5.6 shows the page in the builder, where the fifth tile is visible).

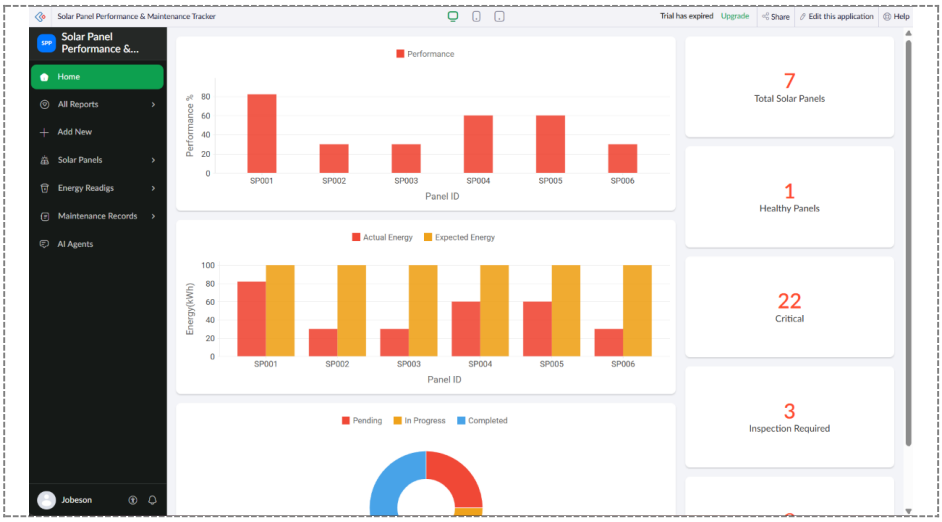


Figure 5.5: Tracker Dashboard in Live Mode

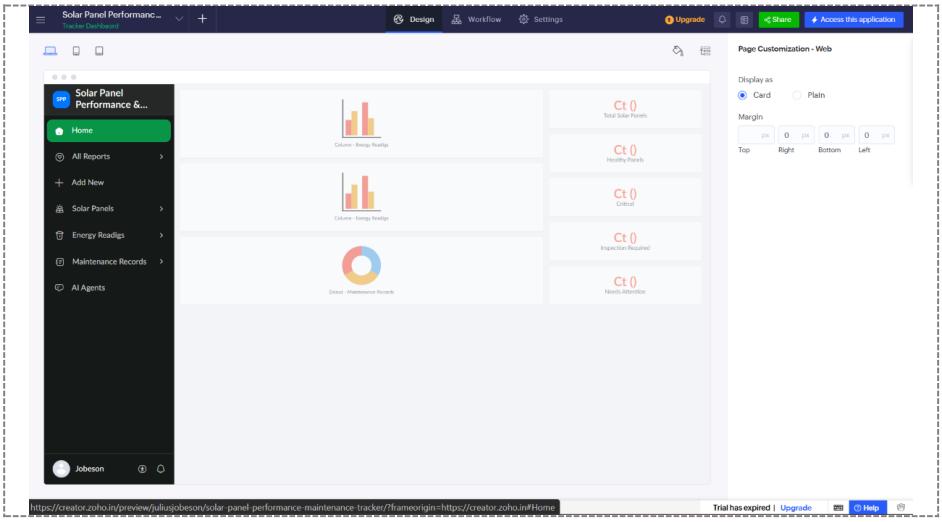


Figure 5.6: Tracker Dashboard in the Page Builder

In the live screenshot the tiles read 7 (Total Solar Panels), 1 (Healthy Panels), 22 (Critical) and 3 (Inspection Required); the Needs Attention tile is cut off. The Performance chart shows about 82 per cent for SP001, about 60 per cent for SP004 and SP005 and about 30 per cent for SP002, SP003 and SP006, against an expected energy of 100 for each. The Critical tile (22) is larger than the number of panels (7), so it appears to count something other than panels, for example readings; the tile's criteria should be checked (Table 6.2, D-02).

The project description also lists an average performance percentage on the dashboard; it is not visible in the screenshots supplied.

5.5 Deluge Scripting

Table 5.2 lists the automation items that can be identified from the workflow screenshots and the project description. The Deluge source of these items was not supplied. Appendix C therefore gives reference implementations written from the described design, and the only logic reproduced here as the project's own is the formula stated in the description (Code 5.1).

Table 5.2: Deluge Scripts Identified

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Energy Readigs → Calculate Performance (successful form submission, 1 action)	Calculate performance and classify the panel	Reference implementation, Appendix C
DS-02	Energy Readigs → Low Performance Alert (successful form submission, 1 action)	Notify when performance is below 70%	Reference implementation, Appendix C
DS-03	Maintenance Records → Maintenance Reminder (successful form submission, 1 action)	Maintenance reminder; behaviour not described	Reference implementation, Appendix C
DS-04	Self Service Form → show/hide fields (field rules, 7 actions)	Show or hide fields; purpose not described	Field rules
DS-05	Custom API Fetch Solar Weather Data	Retrieve temperature, humidity, cloud cover and solar radiation from Open-Meteo	Reference implementation, Appendix C

Code 5.1: Formula stated for the Calculate Performance workflow

```
// Formula stated in the project description for the "Calculate
    Performance"
// workflow (Energy Readigs, on successful form submission).
// The Deluge source of the workflow itself was not supplied.
Performance = (Actual_Energy / Expected_Energy) * 100;
```

5.6 Workflows and Automation

Table 5.3 lists the four form workflows seen. All are enabled and were created on 03 October 2026. No schedule, approval, Blueprint or batch workflow is visible in the screenshots, which show only the Form workflows tab.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Calculate Performance	Form	Energy Readigs – created or edited, successful form submission	Not shown	1 action	DS-01
Low Performance Alert	Form	Energy Readigs – created or edited, successful form submission	Performance below 70% (from the description)	1 action	DS-02
Maintenance Reminder	Form	Maintenance Records – created or edited, successful form submission	Not shown	1 action	DS-03
show/hide fields	Form	Self Service Form – created or edited, field rules	Not shown	7 actions	DS-04

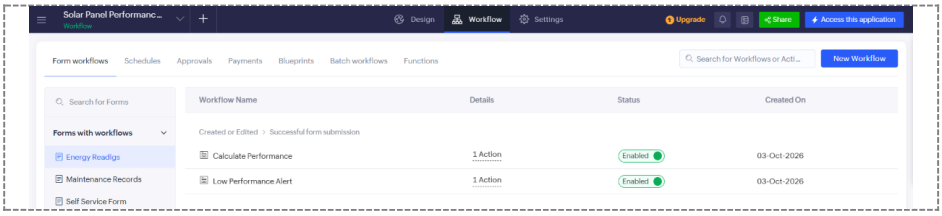


Figure 5.7: Form Workflows of Energy Readigs

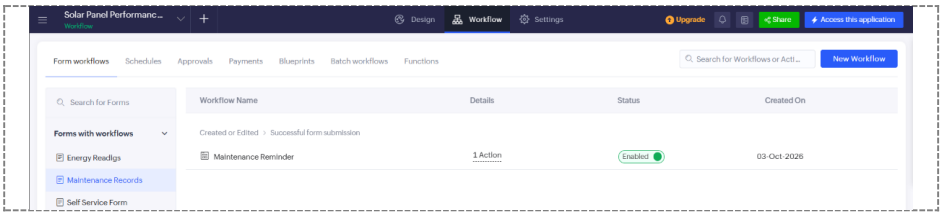


Figure 5.8: Form Workflow of Maintenance Records

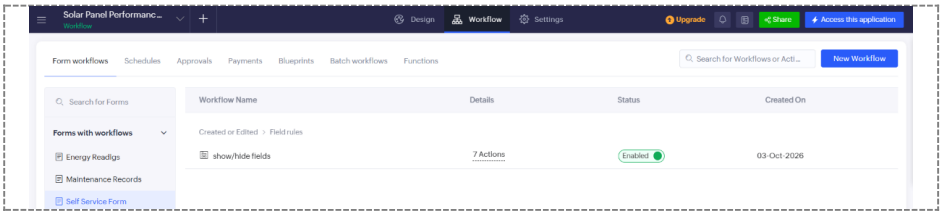


Figure 5.9: Form Workflow of Self Service Form

5.7 Integrations

Table 5.4 lists the integrations. The Microservices area of the Creator home page holds one connection, one AI model and the Custom API list (Figures 5.10 and 5.11). The Custom API Fetch Solar Weather Data uses the GET method and a Public Key, and its list entry reads "Ready to invoke"; a second entry with the same name is shown in Draft state. The description states that the API was tested and returned live weather data; the response itself was not captured. Open-Meteo documents temperature_2m, relative_humidity_2m, cloud_cover and shortwave_radiation as hourly variables [4].

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Weather API	Open-Meteo through Custom API Fetch Solar Weather Data	GET	Public Key	Temperature, relative humidity, cloud cover, solar radiation
Connection	Solar Analytics Connection (solar_analytics_connection)	Not recorded	Status: Connected	The name suggests Zoho Analytics; purpose not confirmed
AI agent	Solar Performance Analyst (solar_performance_analyst)	Zoho AI model	Inside Zoho	Analysis and recommendations
Analytics	Zoho Analytics workspace (Free Edition)	Import from Zoho Creator	Zoho account	Energy and maintenance data analysis

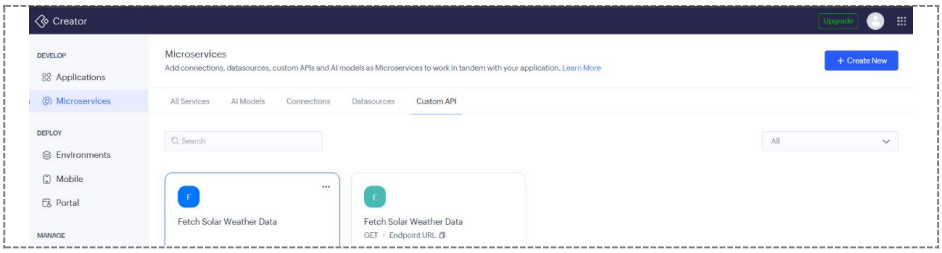


Figure 5.10: Custom API List Showing Fetch Solar Weather Data

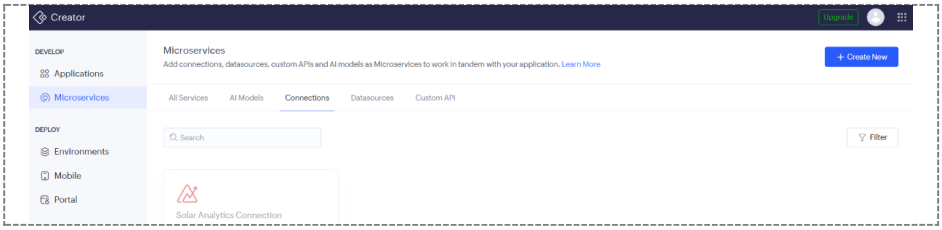


Figure 5.11: Connections List Showing Solar Analytics Connection

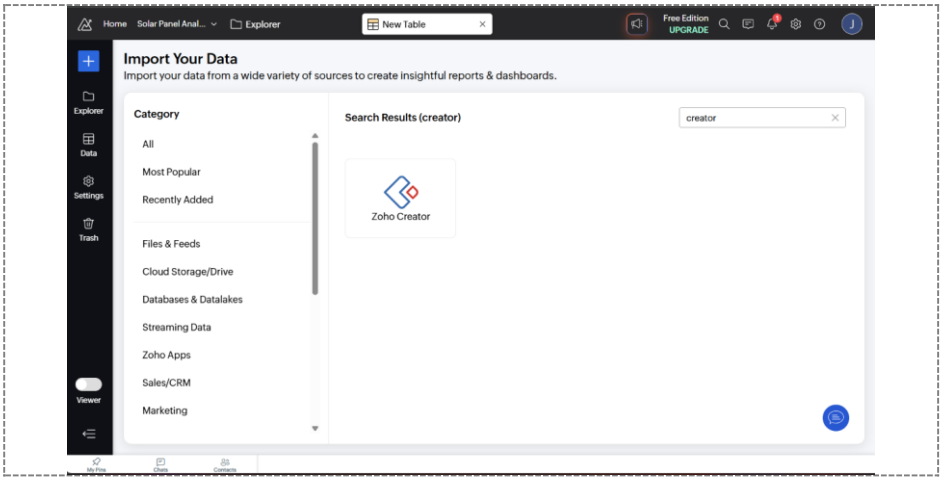


Figure 5.12: Zoho Analytics Import Screen with Zoho Creator Source

The Zoho Analytics screenshot shows a workspace whose Import Your Data screen was searched for the Zoho Creator source. No imported table, report or dashboard from Zoho Analytics was supplied, so the analytics integration is described as started, not completed. A reference request format from the Open-Meteo documentation is given in Appendix C; it is not the project's own script.

5.8 User Management and Role-Based Access

No permission profiles, roles or user invitations were described or shown. The application header offers a Share option, but its use was not recorded. Roles and permission profiles are listed as future work in Section 9.2.

5.9 AI and Analytics Features

The Solar Performance Analyst is defined under Microservices → AI Models as an AI Agent in the Added state (Figure 5.13). It is designed to analyse energy generation, identify critical panels, examine maintenance history, consider environmental conditions and generate maintenance recommendations. The forms hold two places for AI output: AI Maintenance Recommendations in Energy Readigs and AI Maintenance Keywords in Maintenance Records.

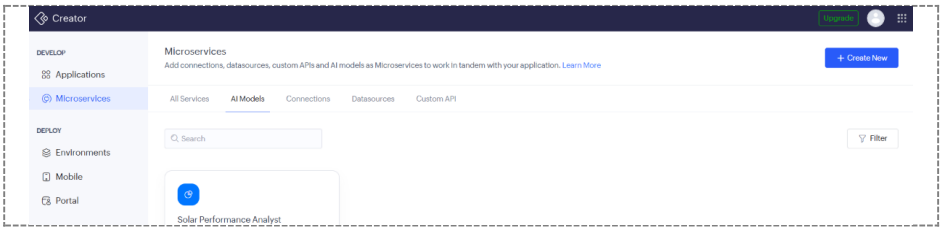


Figure 5.13: AI Models List Showing the Solar Performance Analyst Agent

The AI Agents page of the application (Figure 5.14) shows a chat assistant that greets the user and accepts a request. In the screenshot the assistant displays the message "Chat agent feature is currently disabled for your organization. To enable access, navigate to Operations > Zia from your Zoho Creator homepage." The Zia chat agent therefore could not be used in the state shown, and no conversation or agent output was available to include. How the two AI fields are filled was not recorded. No agent response was available to quote.

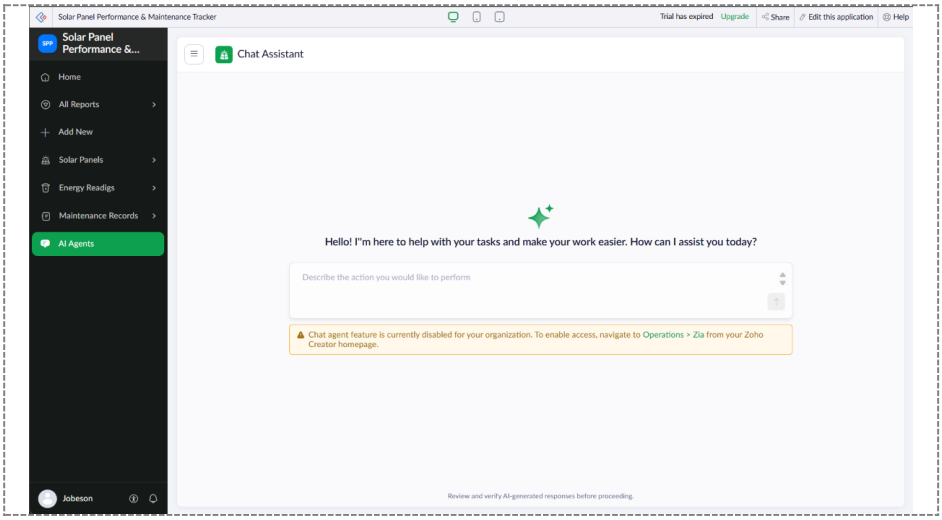


Figure 5.14: AI Agents Page Showing the Chat Assistant Disabled Message

5.10 Pro-Code Extension

No pro-code extension (Catalyst function, widget or external service) is described. The application consists of Zoho Creator components and Deluge.

5.11 Mobile and Portal Access

The application preview shows desktop, tablet and mobile view icons in its header. Use of the Creator mobile app or a customer portal was not described.

5.12 Application Walkthrough

This section follows one session through the application, using the figures above. It describes what a user does and sees; it does not add results that were not captured.

- The user signs in to Zoho Creator and opens the application, which lands on Home, the Tracker Dashboard (Figure 5.5).
- To add a panel, the user opens Solar Panels → Solar Panels and fills the form (Figure 5.2). The panel then appears in All Solar Panels and in the dropdowns of the other two forms.
- To record energy, the user opens Energy Readigs, selects the panel, enters the date, expected energy and actual energy, and submits (Figure 5.3). The Calculate Performance workflow and the Low Performance Alert workflow run on submission (Figure 5.7).
- To record maintenance, the user opens Maintenance Records, selects the panel and fills the problem, technician, cost, action, status and next date (Figure 5.4). The Maintenance Reminder workflow runs on submission (Figure 5.8).
- The user returns to Home, where the charts and tiles reflect the stored records (Figure 5.5).
- The user opens AI Agents to ask a question. In the screenshot the chat feature is disabled for the organisation (Figure 5.14).
- A caller with the Public Key can call Fetch Solar Weather Data to obtain weather values (Figure 5.10).

5.13 Sample Data and Worked Calculation

The dashboard charts show six panel identifiers. Table 5.5 lists the values that can be read from them. The values are approximate readings of bar heights, not exported data, and the expected energy of 100 kWh is the height of the expected-energy bars.

Table 5.5: Values Read from the Dashboard Charts

Panel ID	Expected (kWh)	Actual (kWh)	Performance (%)	Check
SP001	100	about 82	about 82	82 ÷ 100 × 100 = 82
SP002	100	about 30	about 30	30 ÷ 100 × 100 = 30
SP003	100	about 30	about 30	30 ÷ 100 × 100 = 30
SP004	100	about 60	about 60	60 ÷ 100 × 100 = 60
SP005	100	about 60	about 60	60 ÷ 100 × 100 = 60
SP006	100	about 30	about 30	30 ÷ 100 × 100 = 30

The check column applies the formula to the values read from the charts and reproduces the plotted performance in every row. Every panel except SP001 is below 70 per cent, so under the stated rule the Low Performance Alert condition is met for SP002 to SP006. Whether the alert was actually sent was not recorded. The seventh panel counted by the Total Solar Panels tile does not appear in the charts; its identifier and readings were not shown.

5.14 Implementation Status Summary

Table 5.6: Implementation Status Summary

Component	Evidence	Status
Solar Panels, Energy Readigs, Maintenance Records forms	Figures 5.2 to 5.4	Built
Self Service Form	Workflow list (Figure 5.9)	Built; purpose not recorded
Calculate Performance, Low Performance Alert, Maintenance Reminder	Figures 5.7 and 5.8	Built and enabled; output not captured
Tracker Dashboard	Figures 5.5 and 5.6	Built; one tile to be checked
Fetch Solar Weather Data Custom API	Figure 5.10	Ready to invoke; test reported
Solar Analytics Connection	Figure 5.11	Connected; purpose not recorded

Component	Evidence	Status
Solar Performance Analyst agent	Figure 5.13	Added; output not captured
Zia chat assistant	Figure 5.14	Disabled for the organisation
Zoho Analytics	Figure 5.12	Workspace and import screen only
Roles and permissions	None	Not defined

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

The project description records one test (the Open-Meteo call) and the screenshots show the application in use. No test plan, test data sheet or UAT record was supplied, so Table 6.1 lists only the checks that are supported by the material, and the author should add the rest. No automated tests were written.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-03	Performance matches the formula	Panel SP001: expected energy 100, actual energy about 82	Performance about 82%	Chart shows about 82% for SP001 (Figure 5.5)	Pass (observed)
TC-02	FR-08	Dashboard shows data	Open Home	Charts and tiles show values	Seven panels, three charts and tiles shown	Pass (observed)
TC-03	FR-10	Weather fetch through the Custom API	Call Fetch Solar Weather Data	Live weather data returned	Reported as successful; response not captured	Pass (reported)
TC-04	FR-12	Chat assistant answers a question	Open AI Agents page	Assistant replies	Disabled message shown (Figure 5.14)	Fail
TC-05	FR-05	Low-performance alert below 70%	Submit a reading below 70%	Notification raised	Not executed	Not run
TC-06	FR-04	Classification	Submit readings in each range	Correct class set	Not executed	Not run
TC-07	FR-07	Maintenance reminder	Submit a maintenance record	Reminder sent	Not executed	Not run

6.3 Debugging Deluge Scripts

No Deluge error messages or debugging sessions were supplied, so none is described. The checks in Section 4.11 are intended to prevent the errors most likely in these workflows, namely division by zero and a missing lookup value.

6.4 Defect Log

Table 6.2 lists the issues that are visible in the screenshots. They are observations, not a full defect history.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
D-01	Chat assistant disabled.	The message states the chat agent feature is disabled for the organisation.	Enable it under Operations → Zia, as the message advises (not recorded as done).	Open
D-02	Critical tile (22) exceeds the number of panels (7).	The tile appears to count readings or another record type.	Check the tile's source and criteria; count panels, or relabel the tile	Open
D-03	Two Custom API entries share the name Fetch Solar Weather Data, one in Draft.	A draft copy was left behind.	Delete the draft or publish it	Open
D-04	Form and menu spelled "Energy Readigs".	Spelling slip in the form name.	Rename the form to Energy Readings	Open
D-05	Application trial expired.	Trial period ended.	Upgrade or re-create the application in a new account.	Open

6.5 User Acceptance Testing

No user acceptance testing was recorded. The planned tests in Table 6.3 should be run, with at least one other user, before the application is accepted.

6.6 Planned Test Cases

The tests in Table 6.3 are a test plan for the checks that the project material does not record. They have not been run; the Status column says Planned, and the author should fill in the actual result and status when each is executed.

Table 6.3: Planned Test Cases (Not Yet Executed)

TC ID	Area	Input / Steps	Expected Result	Status
TC-08	Performance	Expected 100, actual 100	Performance 100	Planned
TC-09	Performance	Expected 200, actual 150	Performance 75	Planned
TC-10	Performance	Expected 100, actual 0	Performance 0; panel Critical	Planned
TC-11	Performance	Expected 0, actual 50	Rejected or handled without an error	Planned
TC-12	Alert	Performance exactly 70	No alert (condition is below 70)	Planned
TC-13	Alert	Performance 69.9	Alert raised	Planned
TC-14	Alert	Edit an existing low reading	Behaviour on edit recorded (alert repeated or not)	Planned
TC-15	Classification	One reading in each threshold range	Four different status values	Planned
TC-16	Classification	Reading exactly on each threshold	Status as defined in Table 4.3	Planned
TC-17	Solar Panels	Add a panel with all fields	Saved; appears in All Solar Panels	Planned
TC-18	Solar Panels	Add a second panel with the same Panel ID	Rejected or duplicate noted	Planned
TC-19	Lookup	Open the Panel dropdown in Energy Readigs	All stored panels listed	Planned
TC-20	Maintenance	Add a maintenance record	Saved; reminder workflow runs	Planned
TC-21	Maintenance	Next date before maintenance date	Rejected or noted	Planned
TC-22	Dashboard	Add a reading and reload Home	Charts and tiles update	Planned
TC-23	Dashboard	Compare each tile with the records	Tile counts match the records	Planned
TC-24	API	Call without the Public Key	Call refused	Planned
TC-25	API	Call with the Public Key	Weather values for the four parameters returned	Planned
TC-26	AI	Enable the chat feature and ask for the weakest panel	Answer names the lowest-performing panel	Planned

TC ID	Area	Input / Steps	Expected Result	Status
TC-27	AI	Compare AI recommendation with the stored status	Recommendation does not change the status	Planned
TC-28	Views	Open the application on a tablet and a phone view	Forms and dashboard remain usable	Planned

6.7 Test Data Design

Table 6.4 lists the data sets that the planned tests need. The values around 70 per cent are chosen because that is the alert limit; the others are boundary or ordinary cases.

Table 6.4: Test Data Design

Data Set	Expected Energy (kWh)	Actual Energy (kWh)	Purpose
DS-A	100	100	Full performance
DS-B	100	70	On the alert limit
DS-C	100	69.9	Just below the alert limit
DS-D	100	0	Zero generation
DS-E	0	50	Zero expected energy
DS-F	200	150	Different scale, performance 75
DS-G	100	120	Generation above expectation

6.8 Recording Test Results

Each executed test should be recorded with the date, the person who ran it, the input used, the observed result and a screenshot where useful. Failed tests are entered in the defect log (Table 6.2) with a root cause and the fix applied. A test is closed as Pass only when the observed result matches the expected result of the plan.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed and used in a single Zoho Creator instance; development, stage and production environments were not shown. The project description lists the backup components that were prepared: the application structure, solar panel records, energy readings, maintenance records, Deluge functions and workflows, and the AI agent instructions. Table 7.1 records the checklist as far as the evidence goes.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	Application created with forms, workflows and dashboard	✓
2	Weather Custom API created and in the Ready to invoke state	✓
3	AI agent defined and added	✓
4	Backup components prepared (as listed in the description)	✓
5	Chat agent enabled for the organisation	—
6	Publish through development, stage and production environments	—
7	Roles and permission profiles defined	—
8	Draft Custom API entry removed	—

7.2 Performance Optimisation

No performance measurements were taken, so no figures are reported. The design keeps the load small in one respect only: each workflow has one action, apart from the field-rule workflow with seven.

Table 7.2: Performance-Related Design Choices

Design Choice	Component	Measured?	Result	Remarks
One action per calculation, alert and reminder workflow	DS-01 to DS-03	No	Not available	Seen in the workflow list
Dashboard charts read directly from the forms	Tracker Dashboard	No	Not available	Page load time not measured

7.3 Security Features

Table 7.3: Security Controls Identified

Security Control	Where Applied	Threat Mitigated
Public Key on the weather Custom API	Fetch Solar Weather Data	Calls without the key are refused
Zia chat agent disabled at organisation level	AI Agents page	Use of the AI chat feature until the administrator enables it
Sign-in to Zoho Creator	Whole application	Anonymous access to the application

Known gaps: no roles or permission profiles were defined, so every user who can open the application sees all records; a Public Key is a weaker protection than OAuth 2.0 for sensitive data; and the weather data fetched is public information, so exposure through the key is low-risk.

7.4 Governance and Administration

The application is owned by one Zoho Creator account. Workflows are named and enabled, and the application records its creation dates. Audit trail, administration settings and data-retention rules were not described. No governance settings are described; Sections 4.6 and 9.2 list the controls to add.

7.5 Backup and Recovery Procedure

The project description lists the backup components that were prepared. Table 7.4 pairs each with the way it can be saved and restored. The procedure is general; the actual backup files were not supplied.

Table 7.4: Backup Components and Recovery

Component	How to Save	How to Restore
Application structure	Export the application definition from Zoho Creator	Import the definition into the account
Solar panel records	Export the report to a spreadsheet file	Import the file into the form
Energy readings	Export the report to a spreadsheet file	Import after the panels; check the Panel lookup
Maintenance records	Export the report to a spreadsheet file	Import after the panels; check the Panel ID lookup

Component	How to Save	How to Restore
Deluge functions and workflows	Included in the application definition; copy the scripts into Appendix C	Restored with the definition; re-enable if disabled
AI agent instructions	Copy the agent's instructions into a document	Re-create the agent under Microservices and paste them
Custom API and connection	Note names, method and authentication	Re-create under Microservices

Microservices items (Custom API, connection, AI model) sit outside the application definition and must be recreated separately, so their settings should be written down.

7.6 Risk Register

Table 7.5: Deployment and Operation Risks

ID	Risk	Action	Owner
R-01	Trial expiry removes access	Export and upgrade or migrate	Application owner
R-02	Chat agent remains disabled	Enable under Operations → Zia	Application owner
R-03	Draft Custom API remains	Delete or publish the draft	Application owner
R-04	Wrong tile counts	Correct tile criteria	Application owner
R-05	No access control	Define roles and profiles	Application owner
R-06	Weather service unavailable	Handle a failed call without stopping the application	Application owner

7.7 Operational Checklist

Table 7.6 is a routine for whoever runs the application. It is a recommendation for operation, not a record of what was done.

Table 7.6: Recommended Operational Checklist

Frequency	Task	Where
Each reading period	Enter readings for every panel	Energy Readigs
Weekly	Review panels classified Critical or Inspection Required	Dashboard
Weekly	Check that alerts were received for readings below 70%	Notification channel
Monthly	Review maintenance records due and overdue	All Maintenance Records

Frequency	Task	Where
Monthly	Export data and the application definition	Reports and application menu
As needed	Check Custom API status and key	Microservices → Custom API

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The dashboard is the visible result of the application (Figure 5.5). Table 8.1 lists the values read from it.

Table 8.1: Values Observed on the Tracker Dashboard

Item	Value Observed
Total Solar Panels	7
Healthy Panels	1
Critical	22 (larger than the number of panels; see Section 5.4)
Inspection Required	3
Needs Attention	Not visible in the screenshot
Performance by panel (chart)	SP001 about 82%; SP004 and SP005 about 60%; SP002, SP003 and SP006 about 30%
Energy by panel (chart)	Expected energy 100 for each panel; actual energy equal to the performance values above
Maintenance status (donut)	Pending, In Progress and Completed shown; counts not readable

8.2 Achievement of Objectives

Table 8.2: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Evidence	Status
Store panel information	Solar Panels form	Figure 5.2	Achieved
Record energy and calculate performance	Energy Readigs; Calculate Performance	Figures 5.3, 5.5, 5.7	Achieved
Classify panels	Calculate Performance (reported)	Tiles in Figure 5.5; thresholds not supplied	Partly evidenced
Low-performance notification	Low Performance Alert	Workflow listed; not triggered in evidence	Built, not demonstrated
Maintenance records	Maintenance Records; Maintenance Reminder	Figures 5.4, 5.8	Achieved

Objective (Sec. 1.3)	Implemented Through	Evidence	Status
Dashboard	Tracker Dashboard	Figures 5.5, 5.6	Achieved
Weather data through Custom API	Fetch Solar Weather Data	Figure 5.10; test reported	Achieved (reported)
AI assistance	AI agent; chat assistant	Figures 5.13, 5.14	Agent defined; chat disabled

8.3 Intended Capability versus Evidence

No description of an earlier system was supplied, so a comparison with one is not made. Table 8.3 compares what the project set out to do with what the evidence shows.

Table 8.3: Intended Capability versus Evidence Available

Capability	Intended	Evidence Available
Performance calculation	Automatic on each reading	Formula stated; chart values agree with it
Health classification	Four classes	Class tiles on the dashboard; thresholds not supplied
Alerts and reminders	Automatic	Workflows enabled; no alert output captured
Weather data	Four parameters from Open-Meteo	Custom API ready; test reported; no response captured
AI analysis	Chat and analyst agents	Analyst agent added; chat disabled; no output captured
Analytics	Zoho Analytics visualisation	Workspace and import screen only

8.4 Limitations of the Low-Code Approach

- The chat agent could not be used until the organisation setting is enabled; platform settings can block a feature that the application is designed around.
- A trial account limits how long the application remains available.
- Connections, Custom APIs and AI agents are created outside the application, in Microservices, so they must be recreated separately if the application is moved.
- Workflow logic is spread over several small workflows, which makes it harder to review without the Deluge source.
- AI output depends on instructions and cannot be guaranteed; it should be labelled as advisory.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of the application is configuration: forms, lookups, charts, tiles and the menu. The rules (the performance formula, classification and alert) sit at the low-code level in short workflows. The Open-Meteo interface and the AI agent are configured as microservices. No pro-code layer was needed for what was built. A pro-code layer would become relevant if live inverter data were to be read, or if weather values were to be matched to each reading automatically at scale.

8.6 Interpretation of the Dashboard Results

Reading the charts, one panel (SP001) is near 82 per cent, two (SP004 and SP005) are near 60 per cent and three (SP002, SP003 and SP006) are near 30 per cent. Because every expected value is the same, the energy chart and the performance chart carry the same information. If the tiles are classes of panels, only one panel is Healthy, which agrees with SP001 being the only panel above 70 per cent. The other tile values (Inspection Required 3 and Critical 22) cannot be reconciled with the chart values and the design thresholds of Table 4.3, because the Critical tile (22) exceeds the number of panels; they are listed as an open item.

8.7 Challenges and Lessons

- Platform settings can switch off a feature that the application is designed around, as the chat agent message shows. Settings should be checked before a feature is described as working.
- Microservices (Custom API, connection, AI model) are created outside the application, so the application and its integrations must be documented separately.
- Without recorded thresholds and scripts, a result such as a tile count cannot be fully explained. Documentation should be written at the time the workflow is made.
- Small naming slips, such as the Energy Readigs spelling, spread into menus, reports and charts and are harder to correct later.
- Duplicate draft items, such as the second Custom API entry, should be removed when a final version is published.

8.8 Recommendations

Table 8.4: Recommendations Arising from the Review

Recommendation	Reason	Related Item
Enable the chat agent and record sample answers	Shows the AI objective working	D-01
Check the Critical tile source	Count exceeds the number of panels	D-02
Delete or publish the draft Custom API	Avoids confusion between two entries	D-03
Rename the form to Energy Readings	Correct spelling in all menus	D-04
Record thresholds and scripts	Makes the classification verifiable	Table 4.3
Run the planned tests	Gives evidence for every workflow	Table 6.3
Define roles and profiles	Controls who can change data	Section 4.6

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Solar Panel Performance & Maintenance Tracker brings panel records, energy readings and maintenance history into one Zoho Creator application. It calculates performance from expected and actual energy, classifies panel health, raises a low-performance notification, shows the position on a dashboard, can fetch weather data through a Custom API and has an AI agent defined for analysis and recommendations.

The report states plainly what is not shown: the Deluge sources, the configured thresholds, the alert channel, any AI output and any weather response were not supplied (the code and thresholds given are reference values), the chat assistant was disabled in the account, and the Critical tile needs checking. Chapter 6 lists these as open items.

9.2 Future Enhancements

- Enable the Zia chat agent and capture sample conversations.
- Store the weather values fetched from Open-Meteo against each reading and use them in the AI analysis.
- Define roles and permission profiles for operators, technicians and viewers.
- Correct the Critical tile so that it counts panels, and add a tile for average performance.
- Complete the Zoho Analytics integration with imported tables and reports.
- Write and record test cases for each workflow, including the 70 per cent alert.
- Remove the draft Custom API entry and correct the spelling of the Energy Readings form.

9.3 Learning Outcomes and Career Relevance

The project gave practice in modelling related forms with lookups, expressing a calculation and alert as form workflows, publishing a Custom API, defining an AI agent as a microservice and presenting data on a dashboard. It also showed the value of recording evidence for each feature, and of checking platform settings (such as the chat agent) before relying on a feature. These skills apply to roles in low-code development, business-application support and integration.

9.4 Enhancement Roadmap

Table 9.1 groups the enhancements in the order in which they depend on one another.

Table 9.1: Enhancement Roadmap

Step	Enhancement	Depends On
1	Correct the open items D-01 to D-05	Administrator access
2	Record thresholds and Deluge sources	Author's own records
3	Run the planned tests and record results	Step 2
4	Store weather values with each reading	Working Custom API
5	Add roles and permission profiles	Decision on user groups
6	Import data into Zoho Analytics and build reports	Working connection
7	Use weather and maintenance history in AI recommendations	Steps 4 and chat agent enabled

Each step can be added without changing the data model of Chapter 4, apart from the new field or form needed to hold weather values in step 4.

REFERENCES

[1] A. Bock and U. Frank, "Low-Code Platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[2] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in *Proc. Euromicro Conf. on Software Engineering and Advanced Applications (SEAA)*, 2020, doi: 10.1109/SEAA51224.2020.00036.

[3] Zoho Corporation, "Creating and Managing Custom APIs," Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/microservices/custom-api/articles/create-and-manage-custom-apis (Accessed: 01 November 2026).

[4] Open-Meteo, "Weather Forecast API," Open-Meteo Docs. [Online]. Available: https://open-meteo.com/en/docs (Accessed: 01 November 2026).

[5] Zoho Corporation, "Understanding AI Agent," Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-

guide/microservices/ai-modeler/articles/understanding-ai-agent (Accessed: 01 November 2026).

[6] Zoho Corporation, “Understanding and configuring chat agent,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/agents/articles/understanding-and-configuring-chat-agent (Accessed: 01 November 2026).

[7] International Electrotechnical Commission, *IEC 61724-1:2017, Photovoltaic system performance – Part 1: Monitoring*, Edition 1.0, March 2017. [Online]. Available: https://standards.iteh.ai/catalog/standards/iec/6ae4a07c-3eea-4cfe-b563-90ec730e971a/iec-61724-1-2017 (Accessed: 01 November 2026). The catalogue page lists a later revision, IEC 61724-1:2021.

[8] A. Walker, J. Desai and D. Heimiller, *Performance of Photovoltaic Systems Recorded by Open Solar Performance and Reliability Clearinghouse (oSPARC)*, NREL/TP-5C00-75162, National Renewable Energy Laboratory, Golden, CO, USA, Feb. 2020. [Online]. Available: https://www.nrel.gov/docs/fy20osti/75162.pdf (Accessed: 01 November 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Solar Panel Performance & Maintenance Tracker; seven-item menu	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	N	About six types seen (text, dropdown, decimal, date, multi-line, lookup)	4.3	Table 4.2
II	Lookup field (relationship between forms)	Y	Panel and Panel ID lookups	4.2	Fig. 4.2
II	Subform (parent–child data)	N	Not seen	4.2	–
II	Field validations / layout rules	N	Field rules exist in Self Service Form only	5.6	Fig. 5.9
II	List report with filter, grouping, custom action	N	Report details not shown	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	N	Not seen	5.3	–
II	Pivot table / chart report	Y	Column and donut charts	5.4	Fig. 5.5
II	Page / dashboard built with UI Builder	Y	Tracker Dashboard	5.4	Fig. 5.6
II	Deluge – form events (validate / success / field actions)	Y	Successful-submission workflows	5.6	Table 5.3
II	Deluge – functions, fetch, loops, collections	N	Source not supplied	5.5	Appendix C
II	Email / in-app notifications	Y	Low Performance Alert (channel not stated)	5.6	Table 5.3
II	Form workflow	Y	Four enabled form workflows	5.6	Fig. 5.7–5.9
II	Scheduled workflow	N	Not seen	5.6	–
II	Approval process	N	Not seen	5.6	–
II	Blueprint	N	Not seen	5.6	–

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Batch / report workflow	N	Not seen	5.6	–
II	Testing & debugging evidence	N	Limited; see Chapter 6	6	Table 6.1
III	Integration with a Zoho service	Y	Zoho Analytics workspace; Zoho AI agent	5.7, 5.9	Fig. 5.12
III	Connection / REST API call (invokeurl)	Y	Solar Analytics Connection	5.7	Fig. 5.11
III	Webhook or Creator REST API	Y	Custom API Fetch Solar Weather Data	5.7	Fig. 5.10
III	Users, roles and permission profiles	N	Not shown	5.8	–
III	Record-level access criteria / customer portal	N	Not shown	5.8	–
III	Zia / AI feature	Y	AI agent defined; chat assistant disabled	5.9	Fig. 5.13, 5.14
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts; Analytics workspace	5.4, 5.7	Fig. 5.5
IV	Performance optimisation applied & measured	N	Not measured	7.2	Table 7.2
IV	Deployment across environments	N	Single instance	7.1	Table 7.1
IV	Security controls (encryption, OAuth, permissions)	N	Public Key on the Custom API only	7.3	Table 7.3
IV	Governance (audit trail, administration)	N	Not described	7.4	–
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	–
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	–
—	Mobile app / portal access	N	Not described	5.11	–

Total features implemented: 13 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.1, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

Listings C.1 to C.4 are reference implementations written from the design described in the project material: the performance formula, the 70% alert limit and the thresholds of Table 4.3. They show how the rules can be coded and are not copies of the scripts in the application. The field link names (Actual_Energy, Expected_Energy, Performance, Status) are assumed. If the scripts in the application differ, replace these listings with the application's own source.

C.1: Calculate Performance – DS-01 (reference implementation)

```
// Reference implementation. Form: Energy Readigs, on successful
    submission.
if(input.Expected_Energy != null && input.Expected_Energy > 0)
{
    perf = (input.Actual_Energy / input.Expected_Energy) * 100;
    perf = perf.round(2);
    if(perf >= 80)
    {
        state = "Healthy";
    }
    else if(perf >= 70)
    {
        state = "Needs Attention";
    }
    else if(perf >= 50)
    {
        state = "Inspection Required";
    }
    else
    {
        state = "Critical";
    }
    rec = Energy_Readigs[ID == input.ID];
    rec.Performance = perf;
    rec.Status = state;
}
```

C.2: Low Performance Alert – DS-02 (reference implementation)

```
// Reference implementation. Form: Energy Readigs, on successful
    submission.
if(input.Performance != null && input.Performance < 70)
{
    sendmail
    [
        from: zoho.adminuserid
        to: zoho.adminuserid
        subject: "Low performance alert: panel " + input.Panel
        message: "Reading dated " + input.Reading_Date + " shows
            performance of "
            + input.Performance + "%, below the 70% limit."
    ]
}
```

C.3: Maintenance Reminder – DS-03 (reference implementation)

```
// Reference implementation. Form: Maintenance Records, on successful
    submission.
if(input.Next Maintenance Date != null)
{
    sendmail
    [
        from: zoho.adminuserid
        to: zoho.adminuserid
        subject: "Maintenance reminder: " + input.Maintenance_ID
        message: "Next maintenance for panel " + input.Panel_ID + " is
due
            on "
            + input.Next_Maintenance_Date + "."
    ]
}
```

C.4: Fetch Solar Weather Data – DS-05 (reference implementation and documented request format)

```
// Request format documented by Open-Meteo [4] for the four parameters
    used.
// This is the documented URL pattern, NOT the project's own script.
https://api.open-meteo.com/v1/forecast
    ?latitude=<LATITUDE>&longitude=<LONGITUDE>

&hourly=temperature_2m,relative_humidity_2m,cloud_cover,shortwave_radiation

// Reference implementation of the function behind the Custom API
    (illustrative).
url = "https://api.open-meteo.com/v1/forecast?latitude=" + lat +
    "&longitude=" + lon
    +

"&hourly=temperature_2m,relative_humidity_2m,cloud_cover,shortwave_radiation"
    + "&forecast_days=1";
resp = invokeurl
[
    url: url
    type: GET
];
return resp;
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application from the Zoho Creator home page (creator.zoho.in) and sign in with the Zoho account that has access to it. The menu on the left lists Home, All Reports, Add New, Solar Panels, Energy Readigs, Maintenance Records and AI Agents.

D.2 Registering a Solar Panel

- Open Solar Panels → Solar Panels.
- Enter Panel ID, Panel Name and Location, choose the Panel Type, enter Capacity and Installation Date, choose the Status, and select Submit.
- Open All Solar Panels to see the stored panels.

D.3 Recording an Energy Reading

- Open Energy Readigs → Energy Readigs.
- Select the Panel, enter the Reading Date, Expected Energy and Actual Energy, and select Submit.
- The Calculate Performance workflow fills in Performance; a reading below 70 per cent triggers the low-performance alert.

D.4 Recording Maintenance

- Open Maintenance Records → Maintenance Records.
- Enter the Maintenance ID, select the Panel ID, describe the Problem, enter the date, Technician, Cost and Action Taken, choose the Status and the Next Maintenance Date, and select Submit.
- Open All Maintenance Records to review the history.

D.5 Reading the Dashboard and Using the AI Agent

- Open Home to see the performance and energy charts, the maintenance status donut and the panel tiles.
- Open AI Agents to use the chat assistant once it has been enabled under Operations → Zia by the administrator.

D.6 Troubleshooting

Table D.1: Troubleshooting Guide

Problem	Likely Cause	What to Do
Chat assistant says the feature is disabled	The organisation setting is off	Ask the administrator to enable it under Operations → Zia
Panel missing from the Panel dropdown	The panel was not saved	Check All Solar Panels and add it
Performance looks wrong	Expected or actual energy entered incorrectly	Open the reading and correct the values
No alert received	Reading was not below 70%, or the channel is not set up	Check the value and the notification settings
Critical tile larger than the panel count	The tile counts something other than panels	Check the tile's source and criteria
Application unavailable	Trial period ended	Upgrade the plan or restore the application elsewhere
Weather API refused	Missing or wrong Public Key	Use the key shown for the Custom API

D.7 Glossary

Table D.2: Glossary of Terms

Term	Meaning
Performance	Actual energy divided by expected energy, multiplied by 100
Expected energy	Energy a panel should generate in the period of the reading
Actual energy	Energy the panel actually generated in that period
kWh	Kilowatt-hour, the unit of energy shown on the dashboard charts
Lookup	A field that links a record to a record of another form
Workflow	Automation that runs when a form is submitted or a rule is met
Deluge	The scripting language of Zoho Creator
Custom API	An endpoint published from the application for outside callers
AI agent	A Zoho microservice that analyses information and answers in plain language
Tile	A card on the dashboard that shows a count

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	JOBESON J	**Register No.**	240701217
Project Title	SOLAR PANEL PERFORMANCE & MAINTENANCE TRACKER	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	**Criterion**	**Indicators**	**Max. Marks**	**Marks Awarded**
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner	**Signature of External Examiner**
Name: _____	Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	JOBESON J	**Register No.**	240701217
Department / Section	CSE	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	**Question / Topic Asked**	**Unit**	**Response (E / G / S / NI)**	**Remarks**
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	**Max. Marks**	**Marks Awarded**
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**
Name: _____	Name: _____